

Operating a Human-Governed Multi-Machine LLM Agent Fleet: An Experience Report

Anton Dziatkovskii 

Platinum Software Development Company, Brisbane, Australia
dzyatkovski.y.a2@gmail.com

July 2026

Abstract

Most published multi-agent LLM systems are single-process orchestrations evaluated on benchmarks. We report on something different: a fleet of LLM agents distributed across five physical machines (an always-on hub, laptops, a family computer, and a VPS anchor), operated continuously for roughly two months (June–July 2026) on real knowledge work by a non-technical founder and his collaborators. The fleet negotiates decisions through a deterministic consensus protocol (*propose* → *counter* → *accept* → *commit* over an append-only, single-writer-per-machine event log), communicates over a dual-rail bus (synced file mailbox plus a group chat that humans also read), enforces an acknowledgement discipline in which silence past an SLA is an incident, and gates every risky action behind a deterministic risk-tier tripwire that escalates to a dedicated human channel. The safety-critical layer makes zero LLM calls: it is auditable file I/O, and we show it derives full fleet state at microsecond cost. This is an experience report, not a benchmark study. Its evidence is (i) a reproducible offline harness — five self-checking scenarios covering the happy path, the human gate, the tripwire, split-brain, and ledger corruption, all passing on commodity hardware — and (ii) a catalog of nine production failure modes, each of which occurred before its guard existed, giving an unusual, historically grounded form of ablation: for every guard we can state what the system actually did without it. We distill the design principles that survived contact with production (single-writer files, dual-rail by construction, delivery is not completion, detect what you cannot prevent, a human gate needs an exit, alert-channel purity, owner-repairability) and state our limitations plainly: this is an $N=1$ longitudinal case study with no comparative baseline. Reference implementation: `claude-consensus` (MIT).

1 Introduction

Run an LLM agent on more than one computer and a new class of problems appears that no amount of model quality fixes. Machines drift apart: a fix lands on one host and the others never hear of it. The human becomes the courier, copy-pasting between terminals, and thereby the single point of failure. “Sent” silently becomes “done”: an agent hands off a task, nobody confirms it landed, and silence is misread as success. And the synchronization link itself dies quietly, muting half the fleet for hours before anyone notices.

The multi-agent LLM literature largely does not address this regime. Frameworks such as AutoGen [1], CrewAI, MetaGPT [2], and LangGraph orchestrate multiple *roles* inside one process or one service; the distribution of agents across physically separate, independently failing machines — each with its own uptime, its own operator, and its own copy of shared state — is left to the reader. Classical distributed consensus (Paxos [5], Raft [6], PBFT [7]) solves a different problem

in kind: agreement on an ordered log among identical deterministic replicas, whereas our agents are heterogeneous, non-deterministic proposers negotiating *semantic* decisions (“adopt this rule,” “migrate this routine to the hub”) where the safety property that matters is not consistency but *no unsanctioned irreversible action*.

This paper reports what it took to run such a fleet in production: five machines, two months of continuous operation (June–July 2026), real work (knowledge-base curation, content pipelines, CRM operations, system administration), a non-technical owner, and a hard requirement that the human is woken up for exactly two reasons — a risky (Tier-2) action or a true deadlock — and nothing else.

Our contributions are those of an experience report:

1. A working architecture for human-governed multi-machine agent coordination: a deterministic consensus core with a risk-tier human gate, a dual-rail message bus with acknowledgement discipline, single-writer append-only state, and layered self-healing for the sync fabric (§3).
2. A reproducible, offline, zero-token evaluation harness — five self-checking scenarios that double as the engine’s integration test — with measured results on commodity hardware (§5).
3. A catalog of nine production failure modes with root causes and the guards they produced (§6). Because every guard was added *after* its incident, the catalog constitutes a historically grounded ablation: for each safety mechanism we can document the behavior of the system without it.
4. Distilled design principles and honest limitations (§7, §8).

The theoretical frame behind the protective design — three antagonistically coupled control circuits from the Gorsky–Anokhin homeostatic lineage — is developed in a companion preprint [13]; a second companion applies the same negative-feedback lens to token economies [14]. This paper is deliberately the practitioner’s side of that pair: what we built, what broke, and what we measured.

2 Related work

Multi-agent LLM frameworks. AutoGen [1] popularized conversational multi-agent orchestration; MetaGPT [2] and ChatDev [3] structure role-played software teams; LangGraph adds graph-structured control flow with human-in-the-loop interrupts; AgentScope [4] supports actor-based distributed deployment. To our knowledge, none packages the combination we found necessary in production: physically distributed consensus with bounded rounds and a leader tie-break, a *deterministic* risk-tier tripwire that force-escalates mislabeled dangerous actions to a dedicated human channel, an acknowledgement/liveness SLA between hosts, and self-healing of the underlying sync fabric. Our claim is emphatically *not* algorithmic novelty; it is that this systems-level integration is what actually keeps a real fleet coherent, and that it is simple enough to run on the standard library alone.

Classical consensus. Paxos [5], Raft [6], and PBFT [7] provide state-machine replication among identical replicas. We borrow their vocabulary (leader, log, split-brain) but solve a smaller and different problem: a handful of trusted-but-fallible machines negotiating semantic proposals at human timescales (minutes to hours, SLA-timed ticks), with the human as the ultimate authority for a defined risk class. Where classical consensus prevents divergence, we often only *detect* it and refuse to let it pass as success (§6D) — a deliberate trade for auditability and owner-repairability.

Agent memory. Long-term memory systems for LLM agents range from tiered self-editing memory (MemGPT [8]) to automatically extracted knowledge graphs with hybrid retrieval (GraphRAG [9], HippoRAG [10], Zep/Graphiti [11], A-MEM [12]). The fleet’s shared memory substrate (§3.7) differs in two pragmatic respects: the knowledge graph is *human-curated* (wikilink structure maintained in the owner’s Obsidian vault) rather than model-extracted, and a small always-loaded *fact ledger* rides in every session’s context at zero retrieval cost, with vector search plus cross-encoder reranking reserved for the long tail. We make no novelty claim for the ingredients; the report is on how this combination behaves under months of real use.

Human oversight of agents. Human-in-the-loop gating for agent tool use is common practice (framework interrupts, permission systems, guardrail toolkits). Our variant is narrow but strict: the gate is *deterministic* (keyword-tier tripwire, not model judgment), it fails *closed* (a mislabeled dangerous action is force-escalated), the approval has a first-class recorded exit path (§6F), and the alert channel is kept free of routine traffic by construction (§6H).

3 The fleet and its coordination layer

3.1 Setting

The fleet comprises five machines: an always-on desktop hub (also the GPU worker), two laptops, a family member’s computer, and a small always-on VPS that acts as an anchor/coordinator. Each machine runs an interactive LLM agent (Claude Code) plus scheduled headless routines. State that must be shared — the knowledge vault, configuration, the message bus — replicates over Syncthing; nothing depends on a central server. The operators are a non-technical founder and two collaborators; “the human” below refers to the founder unless noted.

3.2 Dual-rail bus

Every machine-to-machine message is sent on two rails at once, by construction: (1) a file mailbox inside the synced folder, and (2) a group chat that both machines and humans read. A single entry point (`bus_send.py`) makes single-rail sends impossible. The design intent is that a dead rail becomes a *signal* (the rails disagree) rather than a silent downgrade. Messages carry IDs; every direct order expects an explicit acknowledgement, then a result report. Silence past the SLA (tens of minutes) triggers an automated chase, then escalation — “delivered” is never allowed to masquerade as “done.”

3.3 Single-writer, append-only state

Every writer appends only to its own file (per-machine shards, JSONL). Two machines never write the same file, so whole-file synchronization never conflicts; merge is a read-time operation. This one invariant eliminated the entire class of sync-conflict bugs from the coordination layer, at the cost of a merge-on-read that we show is microsecond-cheap (§5).

3.4 The consensus protocol

Decisions flow through *propose* $\rightarrow$ *counter* $\rightarrow$ *accept* $\rightarrow$ *commit* over the append-only ledger, with a bounded round count, per-step timeouts, a fixed leader for tie-breaks (bounded “disagree-and-commit”), capability-based addressing (a task routes to whichever node can serve it), and split-brain detection on merge. Proposals carry a risk tier. Tier-0/1 (reversible, internal) actions commit

autonomously on agreement or leader tie-break; Tier-2 (money, irreversible, outward-facing, secrets) *cannot* auto-commit: a deterministic tripwire bumps any proposal whose subject contains a dangerous verb (`delete`, `send`, `transfer`, `publish`, ...) to Tier-2 regardless of the label the proposing agent chose, and commit is refused until a human approval is recorded as a first-class ledger event. “Globally done” additionally requires two distinct verifiers, at least one independent of the committer, with a rubber-stamp guard rejecting verbatim-duplicated proofs.

The safety-critical layer — tie-break, the Tier-2 gate, the tripwire, split-brain detection, verification counting — is deterministic standard-library Python with zero LLM calls. The LLM does only what only an LLM can (propose content, judge a counter), *above* the engine. This split is the central safety decision of the design: everything that guards against catastrophe is code you can audit line by line, not model output you have to trust.

3.5 Governance: leader/follower canon and rules-as-files

Shared behavioral rules (the fleet’s “constitution” and operating rules) live as versioned markdown files loaded into every agent’s context. Exactly one machine (the hub) commits changes to this canon; followers receive it read-only and propose amendments through the bus. Local work on any machine stays fully autonomous. This asymmetry ended write-write wars over the rulebook while preserving peer initiative — the same leader-for-canon, peers-for-work split that the consensus layer uses for tie-breaks.

3.6 Self-healing sync

The sync fabric gets three layers of self-heal: a watchdog that alerts on *state change* rather than on every run (alert-fatigue is a safety problem, §6H); a bounded auto-nudge that asks the dead peer’s own scheduled robot to restart its daemon; and an auto-failover that mirrors file-rail messages to the chat rail while a peer is dark. Liveness is judged from live connection state and fresh heartbeats, never from stale file markers.

3.7 The shared memory substrate

The fleet’s long-term memory is the owner’s knowledge vault: thousands of markdown notes with human-curated wikilinks, replicated to all machines. Retrieval is layered by cost: (1) a small always-loaded fact ledger (an index of one-line facts, each backed by a file) rides in every session at zero retrieval cost; (2) deterministic tools (SQL over indexed stores, name search, `grep`) answer what they can for zero tokens; (3) vector search over the curated corpus with cross-encoder reranking retrieves the semantic long tail; (4) the wikilink graph supplies relationship context that embedding similarity misses. The division of labor mirrors the consensus engine’s: deterministic layers do everything they can; the model spends tokens only on judgment and synthesis. We report this substrate as part of the fleet because in practice coordination and memory failures interact — a peer acting on a stale fact is indistinguishable, from the outside, from a peer that never got the message.

4 Evaluation method

An honest evaluation of a production fleet run by its own authors faces obvious epistemic hazards, so we constrain ourselves to two forms of evidence, both checkable by a skeptical reader.

(1) **A reproducible offline harness.** `demo/demo.py` in the published repository runs the *published* reference engine through five scenarios on one host, simulating two machines by switching the machine identity between calls, with the chat rail disabled — no network, no packages, no API key, no LLM. Each scenario self-checks its end-state; the process exits 0 only if all pass, so the demo doubles as the integration test. Scenario A exercises the happy path through double verification; B proves a Tier-2 action cannot auto-commit and that a recorded approval unblocks it; C proves the tripwire force-escalates a dangerous action mislabeled Tier-0; D manufactures a split-brain (both partitions commit) and proves it is caught and escalated on heal, never reported as success; E injects a corrupt line into a shard and proves events after it still parse.

(2) **The failure catalog as historical ablation.** Conventional ablation removes a component and measures degradation. We did not have to simulate this: for each of the nine guards in §6, production *ran without the guard first*, and the incident that resulted is documented (symptom, root cause, fix, code link) in the repository’s failure-mode catalog. This is weaker than controlled ablation — one incident per guard, no counterfactual frequency — and stronger in one respect: the “ablated” condition ran in the real environment, not a sandbox. We present it as what it is.

What we explicitly do *not* claim: comparative performance against other frameworks (none targets this regime, and we have no baseline), statistical significance (this is an $N=1$ longitudinal case), or any measured-adoption evidence beyond our own operation.

5 Measured results

All numbers in this section are from a single run of the public harness on one of the fleet’s laptops (Windows 11, consumer hardware, Python 3.x, cold-ish disk cache), 2026-07-16; the machine-readable output is committed alongside the draft. Absolute milliseconds vary by machine — the claim is the shape, not the digit.

	Scenario	Wall time	CLI calls	Self-checked end-state
A	Happy path	999.8 ms	6	committed; 2 verifiers incl. 1 independent
B	Human gate	769.0 ms	5	auto-escalated; commit refused until approval
C	Tier tripwire	309.0 ms	2	mislabeled Tier-0 stored as Tier-2, escalated
D	Split-brain	782.0 ms	5	dual commit detected as conflict, escalated
E	Corrupt line	656.5 ms	4	events after corrupt line still read
LLM calls across all scenarios:		0	tokens: 0	result: 5/5 pass

Table 1: Harness run, 2026-07-16, fleet laptop. Each step is a fresh Python process; wall time is dominated by interpreter startup, not consensus logic.

Because each CLI verb pays full process startup, the harness also derives state in-process over a realistic ledger: merging all shards and recomputing the state of **50 proposals / 250 events took 1,717.8 μ s per full pass** on this machine — about 6.9 μ s per event, with no incremental cache to invalidate and no index to corrupt, because the engine re-reads the append-only log and re-derives everything each time. In production the verbs run as scheduled ticks every 10–20 minutes, so even the ~ 100 ms process startup is irrelevant. The coordination layer is, for practical purposes, computationally free; the binding constraints are the sync rail (seconds to minutes) and the per-peer SLA timeouts (default 30 minutes) — which is exactly where the engineering effort went.

6 Nine production failure modes

Each entry below occurred in production before its guard existed. We compress to symptom $\rightarrow$ root cause $\rightarrow$ guard $\rightarrow$ lesson; the public catalog carries code links for each.

A. The leader’s infinite self-accept loop. Overnight the leader wrote 17 identical ACCEPT events on its own proposal, one per tick, never converging: an accept only advances state when cast by a non-proposing machine, so the leader’s self-accept was a no-op reissued forever. *Guard:* on timeout of the leader’s own Tier-0 proposal the tie-break is a direct COMMIT, and a self-accept that cannot advance state is never re-cast. *Lesson:* a retry that provably cannot change state is an infinite loop with extra steps; make the terminal action terminal.

B. Tier mis-classification, and the tripwire’s own false positive. The human gate is only as good as the tier label, so a deterministic keyword tripwire force-bumps dangerous verbs regardless of the label. Its first version matched substrings, so the filename `bus_send.py` tripped “send” and force-escalated harmless proposals. *Guard:* word-boundary matching (suffix inflections still match; phrase terms keep substring semantics), removing a false class while keeping every true term. *Lesson:* a safety gate that cries wolf gets disabled by the humans it annoys; precision is a safety property.

C. A corrupt shard line eating everything after it. One malformed line (partial write, sync-conflict junk) silently dropped every subsequent event in that shard; state went stale with no error. *Guard:* parse per line, not per file; a bad line is skipped, its neighbors are read. *Lesson:* in an append-only log fed by unreliable writers, corruption is local — make the reader treat it that way.

D. Split-brain: both partitions commit. During a partition each side saw “agreed,” believed it owned the apply, and committed; healing produced two conflicting COMMITs. *Guard:* two COMMITs from distinct actors on one proposal are detected on merge, marked `conflict` — never silently “committed” — and escalated for human reconciliation with the leader’s log authoritative. *Honest limitation:* this *detects* rather than *prevents*; prevention would need a distributed lock we deliberately did not build, because the owner must be able to reason about the system. *Lesson:* if you cannot cheaply prevent a fault, at minimum refuse to let it masquerade as success.

E. “Done” nobody independently checked. The machine that applied a change also verified it, and the fleet accepted the single self-report. *Guard:* globally-done requires ≥ 2 distinct verifiers, at least one not the committer, plus a rubber-stamp guard rejecting verbatim-duplicated proofs. *Lesson:* verification by the party that did the work is a status update, not a check.

F. Tier-2 proposals stuck escalated forever. A correctly escalated action got its human “yes” in chat — and nothing could carry that yes into the ledger, so the proposal sat escalated permanently. *Guard:* an `approve` verb records the human decision as a first-class event with a required who/where/message-id proof; commit then admits Tier-2 only with a recorded approval. *Lesson:* a human gate needs an exit, not just an entrance; a safety stop with no resume path is a different outage.

G. Same-second events sorted by filename. Two events in the same second sometimes sorted by shard filename rather than causality, deriving a wrong state. *Guard:* microsecond timestamps. *Lesson:* if the ordering key is coarser than the event rate, you are sorting by luck.

H. The deadlock alert drowning in heartbeats. The “a human is needed now” ping shared a channel with routine heartbeats, and the humans missed it. *Guard*: a dedicated human-alert channel reserved for exactly two message types — a Tier-2 approval request and a true deadlock; routine traffic is banned by construction. *Lesson*: an alert channel that also carries noise is not an alert channel.

I. Regressive votes from a restored backup. A machine restored from an older backup recast votes it had already superseded, re-opening settled proposals. *Guard (production)*: a per-machine vote watermark in the synced decisions directory — a new vote must be strictly newer than that machine’s last recorded vote on the proposal; peers re-sync the newer watermark, so restoring one machine cannot roll it back. This guard is documented but intentionally not in the minimal published reference, and we flag that asymmetry explicitly. *Lesson*: the protocol and the fleet operations around it fail differently; publishing honestly means saying which guards belong to which.

7 Design principles that survived production

1. **Single-writer everything.** One file, one writer; merge on read. This is the cheapest consistency mechanism we know of that a non-specialist can audit.
2. **Dual-rail by construction, not by policy.** If sending on one rail is possible, it will happen; make the single entry point do both, and treat rail disagreement as signal.
3. **Delivery is not completion.** Every order expects an ACK and then a result; silence past SLA is an incident. The sender owns the outcome.
4. **Determinism guards; models propose.** Every safety-critical decision (tiers, gates, tie-breaks, split-brain, verification counting) is auditable code with zero LLM calls. Token spend is reserved for judgment.
5. **Detect what you cannot afford to prevent** — and never let the detected fault report success (split-brain, D).
6. **A human gate needs an exit (F) and a pure channel (H).** Both halves failed in production before we learned this.
7. **Fail-closed tripwires with word-boundary precision (B):** the gate must fire on mislabeled danger and must not fire on filenames, or humans will disable it.
8. **Owner-repairability as a design bound.** Standard library only, no server, no database, no distributed lock: every component must be repairable by a non-technical owner “with a hammer and a screwdriver.” Several stronger designs were rejected on this bound alone.

8 Limitations

This is an $N=1$ case study of one fleet, operated by its authors, over roughly two months; nothing here is a controlled experiment. The harness measures the engine on one host — there is no transport latency in Table 1; real cross-machine convergence is dominated by the sync rail and SLA timers (seconds to tens of minutes). The failure catalog is evidence of guards earned, not

proof of future safety; the published reference is a sanitized subset of the production engine, and we mark production-only guards explicitly. We report no comparative baseline because we know of no framework targeting the same regime, which is itself a claim readers should treat skeptically. Finally, the memory substrate’s contribution is described qualitatively; instrumenting recall quality under fleet conditions is future work, and we would rather say so than invent a number.

9 Conclusion

A fleet of LLM agents across physically separate machines can run real work for months under a discipline simple enough for its non-technical owner to repair: deterministic consensus over single-writer logs, dual-rail messaging with acknowledgement debt, a fail-closed human gate for irreversible actions, and layered self-healing for the fabric underneath. None of the individual mechanisms is novel; the report is that this particular integration holds up, what it cost to learn, and exactly where it still fails. The reference implementation, harness, and failure catalog are public (`claude-consensus`, MIT); the theoretical frame is developed in companion work [13, 14]. We offer the scars as prior art so others do not pay for them twice.

References

- [1] Q. Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.
- [2] S. Hong et al. MetaGPT: Meta programming for multi-agent collaborative framework. arXiv:2308.00352, 2023.
- [3] C. Qian et al. ChatDev: Communicative agents for software development. ACL, 2024.
- [4] D. Gao et al. AgentScope: A flexible yet robust multi-agent platform. arXiv:2402.14034, 2024.
- [5] L. Lamport. The part-time parliament. ACM TOCS 16(2), 1998.
- [6] D. Ongaro, J. Ousterhout. In search of an understandable consensus algorithm. USENIX ATC, 2014.
- [7] M. Castro, B. Liskov. Practical Byzantine fault tolerance. OSDI, 1999.
- [8] C. Packer et al. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.
- [9] D. Edge et al. From local to global: A graph RAG approach to query-focused summarization. arXiv:2404.16130, 2024.
- [10] B. Jimenez Gutierrez et al. HippoRAG: Neurobiologically inspired long-term memory for LLMs. NeurIPS, 2024.
- [11] P. Rasmussen et al. Zep: A temporal knowledge graph architecture for agent memory. arXiv:2501.13956, 2025.
- [12] W. Xu et al. A-MEM: Agentic memory for LLM agents. arXiv:2502.12110, 2025.
- [13] A. Dziahtkovskii. Homeostatic governance: A Gorsky–Anokhin alternative to the Viable System Model for decentralized organizations. Preprint, 2026.

- [14] A. Dziatkovskii. Le Chatelier tokenomics: A thermodynamic–homeostatic framing of negative-feedback token supply control. Preprint, 2026.
- [15] Palo Alto AI Research Lab. claude-consensus: A multi-machine consensus protocol for LLM agent fleets. <https://github.com/tonydzi/claude-consensus>, 2026. MIT license.